Multi-Agent AI Execution and Resource Management System

G.Harsha Charan Reddy	Roll no: 2520030200
M.Jashwanth	Roll no:2520030590
B.Vishnu Vardhan	Roll no:2520030392

Under the Guidance

of

Dr. Y Harika Devi

Assitant Professor

Department of Computer Science and Engineering

(DEEMED TO BE UNIVERSITY)

KONERU LAKSHMAIAH EDUCATION FOUNDATION

(DEEMED TO BE UNIVERSITY)

GROUP NUMBER:1

Name of Candidate	Role
G.Harsha Charan Reddy	AI / Agent Architecture
M.Jashwanth	Backend & Resource Management
B.Vishnu Vardhan	Testing, ML & Documentation

ABSTRACT

- The **Multi-Agent AI Execution and Resource Management System** is an AI-based platform designed to execute complex tasks using multiple specialized agents.
- The system receives a high-level user request and automatically:
- Breaks the task into smaller subtasks.
- Assigns subtasks to suitable AI agents.
- Allocates computational resources.
- Executes tasks using available tools and models.
- Validates the generated results.
- Handles failures and retries.
- Monitors execution and resource usage.
- The main objective is to provide **reliable, scalable and resource-efficient AI task execution**.
- **Key Flow**
- **PLAN → ALLOCATE → EXECUTE → VERIFY → ADAPT**

(DEEMED TO BE UNIVERSITY)

INTRDOUCTION

- Traditional AI systems generally depend on a single model or agent to handle an entire user request.
- However, complex tasks may require:
- Multiple types of reasoning.
- Different tools and models.
- Parallel task execution.
- Large computational resources.
- Result verification.
- Our proposed system uses a **multi-agent architecture**, where each agent specializes in a particular activity.
- A central **Orchestrator** coordinates these agents and manages the overall workflow.
- The system also includes a **Resource Manager** that monitors and allocates resources such as CPU, memory, GPU, API access and execution time.

EXISTING SYSTEM

- **Limitations of Existing Systems**
- Single-agent systems handle most activities in one workflow.
- Complex tasks require manual task decomposition.
- Resource allocation is often static.
- Different agents may compete for the same resources.
- Failure of one task can affect the complete workflow.
- Limited monitoring of agent activities.
- Difficult to trace decisions and execution steps.
- Scaling the system for multiple tasks can be challenging.
- **Main Gap**
- There is a need for a unified system that combines:
- **Agent Coordination + Task Execution + Resource Management + Monitoring**

PROBLEM

- How can we execute complex AI tasks using multiple specialized agents while dynamically managing limited computational and service resources?
- **Major Challenges**
- **Task Decomposition**
  Breaking a complex request into smaller subtasks.
- **Agent Selection**
  Selecting the most suitable agent for each task.
- **Resource Allocation**
  Efficiently allocating CPU, GPU, memory and API resources.
- **Task Coordination**
  Managing dependencies between different subtasks.
- **Failure Recovery**
  Retrying failed tasks without restarting the entire workflow.
- **Monitoring**
  Tracking task progress, performance, latency and resource consumption.
- **Goal**
- To create an **efficient, reliable and scalable multi-agent execution system**.

SOLUTION(Contribution)

- **Proposed Solution**
- The proposed system consists of several cooperating components.
- **1. User Request**
- Receives the task or objective from the user.
- **2. Orchestrator**
- Controls the complete workflow and maintains task state.
- **3. Planner Agent**
- Decomposes the user request into smaller tasks and establishes dependencies.
- **4. Resource Manager**
- Allocates available resources according to task requirements and priority.

5. Specialized Agents

Different agents execute different types of tasks.

Examples:

Research Agent

Execution Agent

Analysis Agent

Validation Agent

6. Shared State / Memory

Maintains task information and intermediate results.

7. Monitoring

Records execution logs, failures, latency and resource usage.

Overall Architecture

User → Orchestrator → Planner → Resource Manager → Agents → Validation → Final Response

Objectives

- **1. Task Planning**
  Automatically divide complex tasks into manageable subtasks.
- **2. Agent Coordination**
  Assign each task to an appropriate specialized agent.
- **3. Resource Management**
  Efficiently manage CPU, GPU, memory, APIs and execution budgets.
- **4. Reliable Execution**
  Provide retries, validation and recovery mechanisms.
- **5. Performance Monitoring**
  Monitor latency, failures, task status and resource consumption.
- **6. Scalability**
  Allow new agents and tools to be added easily.
- **7. Cost Efficiency**
  Reduce unnecessary resource and model usage.

Modules

- **M1 — System & AI Execution Lifecycle**
- The complete workflow is:
- **User Request → Task Planning → Resource Allocation → Agent Execution → Verification → Response → Monitoring**
- The system continuously observes execution and can improve future decisions.
- **M2 — Linear Models & Baselines**
- Classical ML models can be used for simple prediction and decision-making tasks.
- Applications include:
- Resource demand prediction.
- Task priority prediction.
- Agent selection.
- Workload classification.

Techniques include:
Logistic Regression.
Ridge/Lasso.
Feature scaling.
Categorical encoding.
Missing-value handling.

M3 — Tree-Based Models

Tree-based models can be used for structured workload and resource-management decisions.
Examples:
Decision Trees.
Random Forest.
XGBoost.
LightGBM.
They can help predict:
Resource requirements.
Task execution time.
Workload categories.
Agent suitability.
Feature importance can also help explain model decisions.

M4 — Unsupervised Learning

Unsupervised learning can identify patterns in workloads without predefined labels.

Applications:

Grouping similar tasks.

Identifying workload types.

Detecting unusual resource usage.

Finding abnormal execution patterns.

Techniques:

K-Means.

Hierarchical Clustering.

DBSCAN.

PCA.

Isolation Forest.

Cluster information can also be used as features for downstream prediction.

M5 — Model Evaluation & Selection

ML components need to be evaluated before deployment.

Important techniques include:

Train / Validation / Test split.

K-Fold Cross Validation.

Stratified Cross Validation.

Hyperparameter tuning.

Classification metrics:
Accuracy.
Precision.
Recall.
F1 Score.
ROC-AUC.
PR-AUC.
The system can also use probability calibration when decisions depend on predicted confidence.

M6 — ML Engineering & Production

This module focuses on moving ML components from experimentation to production.
Important components include:
Model packaging.
REST APIs using FastAPI.
Batch and online prediction.
Docker-based deployment.
Monitoring.
Data drift detection.
Model retraining.
Experiment tracking.
Data and model versioning.

SOFTWARE REQUIREMENTS

- **Programming**
- Python 3.x
- Object-Oriented Programming
- REST API development
- **AI / ML**
- Large Language Model / AI Model API
- Scikit-learn
- XGBoost / LightGBM
- Feature preprocessing
- Model serialization
- **Backend**
- FastAPI
- Pydantic
- Asynchronous task execution
- **Database**
- SQLite / PostgreSQL
- **Deployment**
- Docker
- Git
- Virtual Environment
- **Monitoring**
- Application logs
- Execution metrics
- Resource utilization monitoring

CONCLUSION AND FUTURE SCOPE

The **Multi-Agent AI Execution and Resource Management System** provides an organized approach for executing complex AI tasks.
Instead of depending on a single agent, the system distributes work among specialized agents and coordinates their activities through a central orchestrator.
The integration of **task planning, resource management, execution, validation and monitoring** provides a foundation for reliable and scalable AI automation.

FUTURE SCOPE

Intelligent agent selection using ML.
Predictive resource allocation.
Distributed multi-node execution.
Cost-aware task scheduling.
GPU-aware resource management.
Advanced security and access control.
Online learning.
Automatic model retraining.
Real-time monitoring dashboards.
Integration with cloud computing platforms.

THANK YOU